

Framework 7 Kategori untuk Membedah Soal LeetCode

Cara berpikir sederhana: algoritma adalah proses mengubah input menjadi output. Framework ini membantu lu melihat bagian-bagian mesin algoritmanya.

Core idea

LeetCode bukan nebak pattern. Lu isi 7 kategori ini: input shape, state, movement, working structure, decision rule, complexity, edge case. Dari kombinasi itu, pattern biasanya kebaca sendiri.

1. Peta Besar

Kategori	Pertanyaan inti	Jawaban contoh
Input shape	Soal ngasih data bentuk apa?	Array, string, grid, tree, graph, linked list, sorted space, list kategori
State	Variable kondisi sekarang apa?	index, left/right, row/col, node, distance, path, window, low/high/mid
Movement	State berubah gimana?	i++, 4 arah, child node, choose/undo, expand/shrink, split half
Working structure	Alat internal apa yang dipakai?	Queue, stack, set, map, heap, visited, path, result, dp table, trie
Decision rule	Aturan if/else-nya apa?	skip, valid, update, save, return, undo, shrink, prune
Complexity	Mahalnya naik kayak apa?	$O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(V+E)$, $O(2^n)$
Edge case	Kasus pinggir apa yang bisa ngerusak?	empty, duplicate, null, blocked, no answer, cycle, overflow, off-by-one

2. Input Shape

Input shape = bentuk input yang dikasih soal. Ini bukan "data structure teori" secara umum, tapi peran data sebagai input soal.

- Array/List: `nums = [1, 2, 3]`
- String: `s = "abcabcb"`
- Grid/Matrix: `[[0,1],[1,0]]`
- Tree: `root = TreeNode(...)`
- Graph: `edges = [[1,2],[2,3]]` atau adjacency list
- Linked List: `head = 1 -> 2 -> 3`
- Intervals: `[[1,3], [2,6]]`
- Sorted search space: array sorted atau range jawaban `1..1e9`
- List of choices: `[[nasi,mie], [teh,kopi], [puding,es krim]]`

Pertanyaan kunci: "Medan yang dikasih soal bentuknya apa?"

3. State

State = variable-variable yang menggambarkan kondisi algoritma saat ini. Bukan semua variable, tapi variable yang menunjukkan posisi, progress, atau kondisi sementara.

Pattern / kasus	State yang umum	Maknanya
Binary Search	left, right, mid	Batas search space dan titik yang sedang diuji
BFS Grid	row, col, distance	Posisi sekarang dan jarak dari start
Backtracking	level, path	Kategori/posisi sekarang dan pilihan sementara
Sliding Window	left, right, freq	Batas window aktif dan isi window
DFS Tree	current node, depth	Node yang sedang diproses dan kedalamannya
DP	i, j, dp[i][j]	Subproblem yang sedang dihitung

Kalimat gampang: State = "sekarang algoritma gue lagi di kondisi apa?"

4. Movement

Movement = cara state berubah. Movement tidak harus sekadar menelusuri input. Movement nempel ke state: state sekarang berubah menjadi state berikutnya.

Kasus	Movement
Array scan	$i \rightarrow i + 1$
Grid BFS/DFS	(row,col) $\rightarrow$ 4 arah atau 8 arah
Tree DFS	node $\rightarrow$ node.left / node.right
Linked List	current $\rightarrow$ current.next
Backtracking	path.add(choice), level + 1, lalu undo
Sliding Window	right maju untuk expand, left maju untuk shrink
Binary Search	tes mid, lalu left/right berubah
DP	dp[i] dihitung dari state sebelumnya

Movement = bukan "jalan di data" doang. Movement = transisi kondisi.

5. Working Structure

Working structure = tipe data/bentuk data internal yang dibuat di dalam algoritma untuk membantu proses. Bukan input utama, tapi alat kerja sementara.

Kebutuhan	Working structure
Ingat item yang sudah dilihat	Set
Mapping key ke value	Map / HashMap
BFS layer	Queue
DFS iterative / undo / bracket	Stack

Kebutuhan	Working structure
Ambil prioritas min/max	Heap / PriorityQueue
Tandai node/cell sudah diproses	Visited set/matrix
Pilihan sementara	Path list
Semua hasil akhir	Result list
Simpan parent untuk reconstruct path	Parent map
Simpan jawaban subproblem	DP table
Prefix kata	Trie
Gabung/cek kelompok	Union Find

Input = bahan mentah. Working structure = alat kerja internal.

6. Decision Rule

Decision rule = aturan if/else yang menentukan nasib state sekarang atau kandidat next state. Ia mengatur skip, lanjut, update, save, undo, atau stop.

Situasi	Decision rule
Grid neighbor	Keluar grid? skip. Blocked? skip. Visited? skip. Valid? masuk queue.
Binary Search	nums[mid] < target? geser left. > target? geser right. == target? return.
Backtracking	Path lengkap? save. Choice invalid? skip. Selesai explore? undo.
Sliding Window	Window invalid? shrink left. Window valid? update answer.
Heap scheduler	Heap top sudah due? pop dan proses. Belum due? tunggu.

Intinya: current state -> cek aturan -> next state / skip / update / stop.

7. Complexity

Complexity = ukuran biaya algoritma saat input membesar. Ada dua: time complexity dan space complexity.

Complexity	Rasa pertumbuhan	Contoh umum
$O(1)$	Tetap	akses index, beberapa variable
$O(\log n)$	Naik pelan	binary search
$O(n)$	Linear	scan sekali
$O(n \log n)$	Sedang	sorting
$O(n^2)$	Kuadratik	nested loop pasangan
$O(m*n)$	Sebesar grid	traversal matrix
$O(V+E)$	Node + edge	BFS/DFS graph
$O(2^n)$	Meledak subset	subsets/backtracking tertentu

Complexity	Rasa pertumbuhan	Contoh umum
$O(n!)$	Meledak permutasi	semua permutasi

Time = jumlah langkah. Space = memory tambahan. Yang dibaca adalah pola pertumbuhan, bukan detik pastinya.

8. Edge Case

Edge case = kondisi pinggir yang bisa merusak salah satu bagian algoritma. Edge case nempel ke semua kategori: input, state, movement, working structure, decision rule, complexity, output.

Sumber edge case	Contoh
Input shape	array kosong, string kosong, grid kosong, tree null, kategori kosong
State	left > right, start == goal, distance mulai 0 atau 1, path kosong
Movement	keluar batas, blocked, balik ke node yang sama, cycle
Working structure	queue kosong, stack kosong, heap kosong, key map tidak ada, removeLast saat path kosong
Decision rule	off-by-one, lupa visited, lupa undo, duplicate tidak ditangani, valid case malah skip
Complexity	input besar bikin TLE, recursion terlalu dalam, result terlalu besar
Output	tidak ada jawaban return apa, hasil kosong valid atau tidak, hitung node atau edge

Edge case = stress test untuk semua bagian algoritma.

9. Contoh Bedah Cepat

Shortest Path in Binary Matrix

Kategori	Isi
Input shape	Grid 0/1
State	row, col, distance
Movement	8 arah
Working structure	Queue + visited
Decision rule	Keluar grid/1/visited skip. Goal return distance. Valid masuk queue.
Complexity	$O(n^2)$ time, $O(n^2)$ space
Edge case	Start blocked, goal blocked, grid 1x1, tidak ada path

Backtracking Menu

Kategori	Isi
Input shape	List of lists: makanan, minuman, dessert
State	level, path
Movement	Pilih item dari kategori level sekarang, lalu level+1
Working structure	path + result
Decision rule	level selesai -> save path. Setelah explore -> undo
Complexity	perkalian jumlah pilihan tiap kategori
Edge case	kategori kosong, input kosong, duplicate/constraint

Binary Search

Kategori	Isi
Input shape	Sorted array atau ordered answer space
State	left, right, mid
Movement	Tes mid, buang setengah search space
Working structure	Variable boundary saja
Decision rule	mid terlalu kecil -> left naik. Terlalu besar -> right turun.
Complexity	$O(\log n)$ time, $O(1)$ space
Edge case	array kosong, target di ujung, duplicate, infinite loop/off-by-one

Sliding Window

Kategori	Isi
Input shape	String atau array
State	left, right, window/frequency, answer
Movement	right expand, left shrink
Working structure	Set atau frequency map
Decision rule	window invalid -> shrink. valid -> update answer
Complexity	$O(n)$ time, $O(k)$ space
Edge case	string kosong, semua sama, semua unik

10. Cara Pakai Saat Ketemu Soal Baru

- Jangan mulai dari "ini pattern apa?". Mulai dari "input-nya bentuk apa?"
- Cari state: variable apa yang berubah selama proses?
- Tentukan movement: state berubah lewat scan, neighbor, choose/undo, window, atau split half?
- Pilih working structure: butuh map, set, queue, stack, heap, path, visited, dp?
- Tulis decision rule: kapan skip, valid, update, save, return, undo?
- Hitung complexity: berapa kali state diproses dan sebesar apa alat bantu?
- Uji edge case: kosong, satu item, duplicate, blocked, no answer, off-by-one.

Kalimat final: LeetCode = transformasi input menjadi output. Framework 7 kategori ini adalah cara membedah mesin transformasinya.